

Languages and Algorithms for Artificial Intelligence (Third Module) **The Computational Model**

Ugo Dal Lago



ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA



University of Bologna, Academic Year 2023/2024

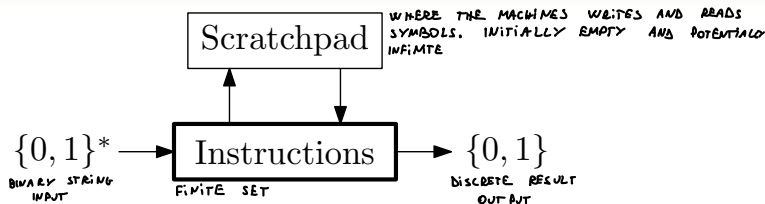
The Need of a Model

- ▶ Giving **positive** results about the feasibility of certain computation task is relatively easy:
 - ▶ You implement your algorithm, you run it on a powerful machine, and that's it!
- ▶ But how about **negative** results?
 - ▶ Which machine should we choose?
 - ▶ If we choose one specific, concrete machine without relating to the other ones, then our negative results would be *vacuous*.
- ▶ The only way out is to define a **model of computation** in the form of an abstract machine, keeping in mind that:
 - ▶ It should be as simple as possible, to *facilitate proofs*.
 - ▶ It must be able to simulate with reasonable overhead *all physically realistic* machines.
- ▶ There is a universally accepted model of computation, that we will take as our reference model, namely the **Turing Machine**.
 - ▶ This part of the module is specifically about it.

Part I

The Model, Informally

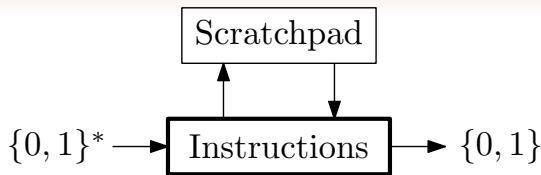
How to Compute $f : \{0, 1\}^* \rightarrow \{0, 1\}$



- ▶ The set of instructions to be followed is fixed (and should work for every input x), and finite.
- ▶ The same instruction can be used potentially many times.
- ▶ Every instruction proceeds by:
 - ▶ Reading a bit of the input;
 - ▶ Reading a symbol from the scratchpad;and based on that decide what to do next, namely:
 - ▶ either write symbol to the scratchpad, and proceed to another instruction;
 - ▶ or **declare the computation finished**, by stopping it and outputting either 0 or 1.

→ may not finish with loops

How to Compute $f : \{0, 1\}^* \rightarrow \{0, 1\}$



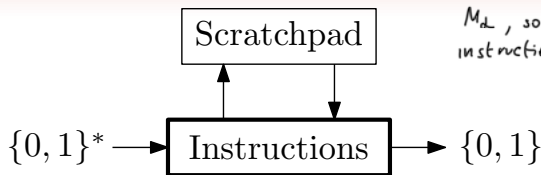
- ▶ The **running time** of this machine/process/algorithm on x is simply the number of these basic instructions which are executed on a certain input x .
- ▶ We say that the machine **runs in time** $T(n)$ if it performs *at most* $T(n)$ instructions on input strings *of length* n .
- ▶ The model is **robust** to many tweaks in the definition, (e.g. changing the alphabet, allowing multiple scratchpads rather than one): **the simplest model can simulate the more complicated with a polynomial overhead in time.**
- ▶ Since there are finitely many instructions, machine descriptions can be **encoded as binary strings** themselves.

How to Compute $f : \{0, 1\}^* \rightarrow \{0, 1\}$

ES:

α : python program

x : input to α



α is a string that encodes the Turing machine M_α , so it is the set of instructions

- ▶ Given a string α , we indicate as M_α the Turing Machine α encodes.
- ▶ There is a so-called Universal Turing Machine $\mathcal{U}$, which simulates any other Turing Machine given its string representation: from a pair of strings (x, α) , the machine $\mathcal{U}$ simulates the behaviour of M_α on x .
 - ▶ The simulation is very efficient: if the running time of M_α were $T(|x|)$, then $\mathcal{U}$ would take time $O(T(|x|) \log T(|x|))$.
- ▶ There are functions $f : \{0, 1\}^* \rightarrow \{0, 1\}$ which are **intrinsically uncomputable** by Turing Machines, and this can be proved formally.
 - ▶ This has intimate connections to Gödel's famous **incompleteness theorem**.

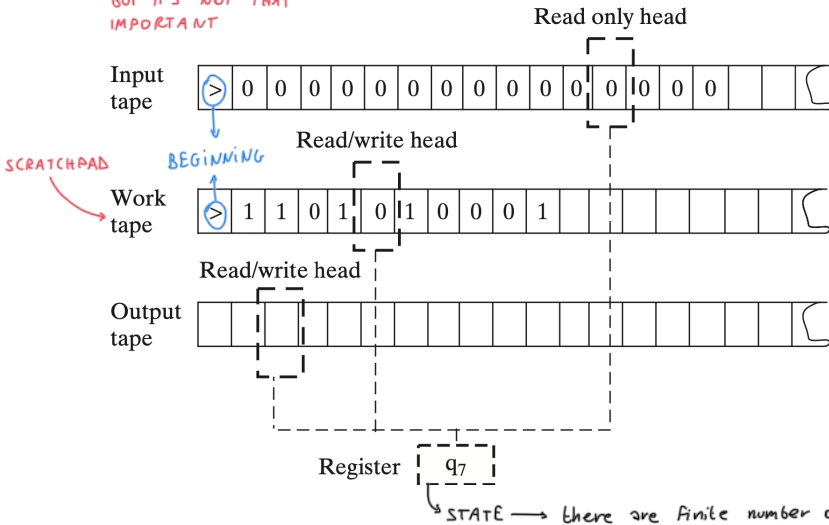
Part II

The Model, Formally

A More Detailed View

THERE IS NO DOGMA ON HOW THEY ARE IMPLEMENTED, FOR EXAMPLE SOME CAN BE ∞ IN BOTH DIRECTIONS OR HAVE R/W PERMISSION EVERYWHERE

SOMETHING CHANGES BUT IT'S NOT THAT IMPORTANT



The Scratchpad(s)

- ▶ It consists of k **tapes**, where a tape is an infinite one-directional line of cells, each of which can hold a symbol from a finite alphabet Γ , the *alphabet* of the machine.
- ▶ Each tape is equipped with **tape head**, which can read or write one symbol at a time *from* or *to* the tape. Each head can move left or right.
- ▶ Some of the tapes can be designated as **input tapes**, and are read-only.
- ▶ The last tape can be taken as the **output tape**, and in that case contains the result of the computation.
 - ▶ This slightly deviates from what we have said in our informal account, and is needed to compute arbitrary functions on $\{0, 1\}^*$.
 - ▶ The output tape(s) can be absent, and in that case the result can be taken as 0 or 1 depending on the *final state*.

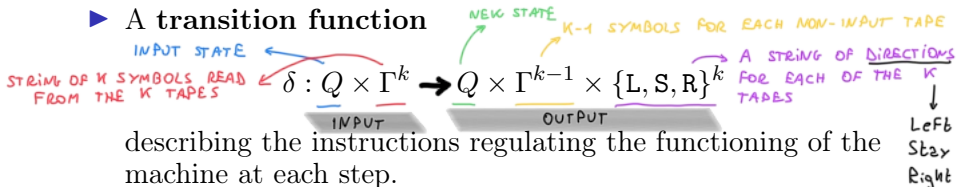
The Instructions

- ▶ The machine has a finite set of *states*, called Q , which determine the action to be taken at the next step.
- ▶ At *each step*, the machine:
 1. **Read** the symbols under the k tape heads.
 2. For the $k - 1$ read-write tapes, **replace** the symbol with a new one, or leave it unchanged.
 3. **Change** its state to a new one.
 4. **Move** each of the k tape heads to the left or to the right (or stay in place).
- ▶ These instructions are of course very basic, and far from being close to the kind of instructions programming languages offer.
 - ▶ The point here is to have a *simple*, but *expressive* model.

The Formal Definition

A **Turing Machine** (TM for short) working on k tapes is described as a triple (Γ, Q, δ) containing

- ▶ A finite set Γ of **tape symbols**, which we assume contains the *blank symbol* $\square$, the *start symbol* $\triangleright$, and the binary digits 0 and 1.
Finite on left AND infinite on right
- ▶ A finite set Q of **states** which includes a designated *initial state* q_{init} and a designated final state q_{halt} .
- ▶ A **transition function**



describing the instructions regulating the functioning of the machine at each step.

- ▶ When the first parameter is q_{halt} , then δ cannot touch the tapes nor the heads:

$$\delta(q_{\text{halt}}, (\sigma_1, \dots, \sigma_k)) = (q_{\text{halt}}, (\sigma_1, \dots, \sigma_k), (S, \dots, S))$$

and the machine is *stuck*.

Machine Configurations and Computations

Given a TM $\mathcal{M} = (\Gamma, Q, \delta)$ working on k tapes:

- ▶ A **configuration** consists of
 - ▶ The current state q .
 - ▶ The contents of the k tapes.
 - ▶ The positions of the k tape heads.

One such configuration will be indicated as C .

- ▶ The **initial configuration** for the input $x \in \{0, 1\}^*$ is the configuration $\mathcal{I}_x$ in which:
 - ▶ The current state is q_{init} .
 - ▶ The first tape contains $\triangleright x$, followed by blank symbols, while the other tapes contain $\triangleright$, followed by blank symbols.
 - ▶ The tape heads are positioned on the first symbol of the k tapes.

Machine Configurations and Computations

Given a TM $\mathcal{M} = (\Gamma, Q, \delta)$ working on k tapes:

- ▶ A **configuration** consists of
 - ▶ The current state q .
 - ▶ The contents of the k tapes.
 - ▶ The positions of the k tape heads.

One such configuration will be indicated as C .

- ▶ The **initial configuration** for the input $x \in \{0, 1\}^*$ is the configuration $\mathcal{I}_x$ in which:
 - ▶ The current state is q_{init} .
 - ▶ The first tape contains $\triangleright x$, followed by blank symbols, while the other tapes contain $\triangleright$, followed by blank symbols.
 - ▶ The tape heads are positioned on the first symbol of the k tapes.
- ▶ A **final configuration** for the output $y \in \{0, 1\}^*$ is any configuration whose state is q_{halt} and in which the content of the output tape is $\triangleright y$, followed by blank symbols.

Machine Computations

Given a TM $\mathcal{M} = (\Gamma, Q, \delta)$ working on k tapes:

- ▶ Given any configuration C , the transition function δ determines in a natural way the next configuration D , and we write $C \xrightarrow{\delta} D$ if this is the case.
- ▶ We say that $\mathcal{M}$ **returns** $y \in \{0, 1\}^*$ **on input** $x \in \{0, 1\}^*$ **in t steps** if

$$\mathcal{I}_x \xrightarrow{\delta} C_1 \xrightarrow{\delta} C_2 \xrightarrow{\delta} \dots \xrightarrow{\delta} C_t$$

where C_t is a final configuration for y . We write $\mathcal{M}(x)$ for y if this holds.

- ▶ Finally, we say that $\mathcal{M}$ **computes** a function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ iff $\mathcal{M}(x) = f(x)$ for every $x \in \{0, 1\}^*$. In this case, f is said to be **computable**.
 - ▶ Beware: for the moment, we do not put any constraint *on the number of steps* $\mathcal{M}$ needs to compute $f(x)$ from x .

The Expressive Power of TMs

- ▶ Please take a look at
`http://turingmachinesimulator.com`
- ▶ Explicitly constructing TMs is **very tedious**.
 - ▶ One needs to give the set of states, the set of instructions, etc.
 - ▶ One also needs to *prove* that the construction is correct.
- ▶ Usually, functions on binary strings are shown to be computable by informally describing *algorithms* or *programs* computing the function.
 - ▶ This is fine, *provided* program or algorithm instructions can be **simulated** by TMs.
- ▶ There are many other formalisms which are perfectly equivalent to TMs as for the class of computable functions they induce.
 - ▶ Examples: Random Access Machines, the λ -calculus, URMs, Partial Recursive Functions, ...

Efficiency and Runtime

- ▶ A TM $\mathcal{M}$ computes a function $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ in **time** $T : \mathbb{N} \rightarrow \mathbb{N}$ iff $\mathcal{M}$ returns $f(x)$ on input x in a number of steps smaller or equal to $T(|x|)$ for every $x \in \{0, 1\}^*$. In this case, f is said to be **computable** in time T .
- ▶ A language $\mathcal{L}_f \subseteq \{0, 1\}^*$ is **decidable** in time T iff f is computable in time T .
- ▶ Examples.
 - ▶ The set of palindrome words is decidable in time $T(n) = 3n$.
 - ▶ Computing the parity of binary strings requires time $T(n) = n + 2$.
 - ▶ Basic operations like addition and multiplication are computable in polynomial time.

On the Robustness of Our Definition

- ▶ Question: *why is our definition the right one?*
- ▶ Actually, there are many details in our definition of a TM which are arbitrary: **many** alternative definitions are available in the literature.
- ▶ Examples:
 - ▶ Rather than an arbitrary tape alphabet Γ , **restrict to** $\{0, 1, \triangleright, \square\}$.
 - ▶ Just **one tape**, rather than many.
 - ▶ Tapes can be infinite **in both directions**.
- ▶ In all the cases above (and in many others), one can prove that the more restrictive notion of machine **simulates** the more general one **with polynomial overhead**.
 - ▶ We do not have time to see all that, but you are encouraged to take a look at [AroraBarak2009], Section 1.3.1.

- ▶ One of the very nice consequences of keeping our definition of a Turing Machine very simple is that any machine $\mathcal{M} = (\Gamma, Q, \delta)$ is in fact completely determined from the graph of δ , seen as a subset of

$$Q \times \Gamma^k \times Q \times \Gamma^{k-1} \times \{L, S, R\}^k$$

- ▶ Any parsimonious encoding of δ as a binary string in $\{0, 1\}^*$ thus constitutes an acceptable encoding $\ulcorner \mathcal{M} \urcorner$ of $\mathcal{M}$, provided the following two conditions are satisfied:
 1. **Every string** in $\{0, 1\}^*$ represents a TM, i.e. for every $x \in \{0, 1\}^*$ there is $\mathcal{M}$ such that $x = \ulcorner \mathcal{M} \urcorner$.
 2. Every TM $\mathcal{M}$ is represented by an **infinitely many strings** (although exactly one is the “canonical” representation $\ulcorner \mathcal{M} \urcorner$ of $\mathcal{M}$).
- ▶ The two conditions above are not essential in any other contexts (i.e. when the encoded data is not a program), but are technically crucial here.

The Universal Turing Machine

Theorem (UTM, Efficiently)

There exists a TM $\mathcal{U}$ such that for every $x, \alpha \in \{0, 1\}^$, it holds that $\mathcal{U}(x, \alpha) = \mathcal{M}_\alpha(x)$, where $\mathcal{M}_\alpha$ denotes the TM represented by α . Moreover, if $\mathcal{M}_\alpha$ halts on input x within T steps then $\mathcal{U}(x, \alpha)$ halts within $CT \log(T)$ steps, where C is independent of $|x|$ and depending only on $\mathcal{M}_\alpha$.*

- ▶ A proof of the Theorem above in its full generality requires quite a bit of work.
- ▶ More specifically, one has to encode configurations of Turing machines as strings, and prove that $\mathcal{U}$ can simulate $\mathcal{M}_\alpha$ for every α .

Uncomputability

Theorem (Uncomputable Functions Exist)

There exists a function $uc : \{0, 1\}^ \rightarrow \{0, 1\}^*$ that is not computable by any Turing Machine.*

- ▶ The proof of the Theorem above is constructive. It suffices to consider the function

$$uc(\alpha) = \begin{cases} 0 & \text{if } \mathcal{M}_\alpha(\alpha) = 1 \\ 1 & \text{otherwise} \end{cases}$$

- ▶ Indeed, if uc were computable, there would be a TM $\mathcal{M}$ such that $\mathcal{M}(\alpha) = uc(\alpha)$ for every α , and in particular when $\alpha = \ulcorner \mathcal{M} \urcorner$.
- ▶ This would be a contradiction, because by definition

$$uc(\ulcorner \mathcal{M} \urcorner) = 1 \Leftrightarrow \mathcal{M}(\ulcorner \mathcal{M} \urcorner) \neq 1 \Leftrightarrow uc(\ulcorner \mathcal{M} \urcorner) = 0$$

The Halting Problem

- ▶ The function *uc* we proved uncomputable does not represent an interesting computational task.
- ▶ Consider, instead, the function *halt* defined as follows:

$$\text{halt}(\ulcorner(\alpha, x)\urcorner) = \begin{cases} 1 & \text{if } \mathcal{M}_\alpha \text{ halts on input } x; \\ 0 & \text{otherwise.} \end{cases}$$

- ▶ Being able to compute *halt* would mean being able to check algorithms for termination.

Theorem (Uncomputability of *halt*)

*The function **halt** is not computable by any TM.*

- ▶ This result could be seen as a way to reinterpret Gödel's first incompleteness theorem “computationally”.

Diophantine Equations

- ▶ A **diophantine equation** is a polynomial equality with integer coefficients and finitely many unknowns.
- ▶ **Examples:**

$$x^2 + 3y = 2x + 1 \qquad x^4 + 3x^3 + y - z = 12$$

Theorem (MDPR Theorem)

The problem of determining whether an arbitrary diophantine equation has a solution is undecidable.

Rice's Theorem

If the language is about the **IMPLEMENTATION** and so if two machine solve the same problem then they come from the same language

- ▶ A language $\mathcal{L} \subseteq \{0,1\}^*$ is **semantic** when the following condition holds: if $\mathcal{M}$ and $\mathcal{N}$ compute the same function, then $\lfloor \mathcal{M} \rfloor \in \mathcal{L}$ if and only if $\lfloor \mathcal{N} \rfloor \in \mathcal{L}$.
- ▶ Semantic languages can be seen as extensional properties of programs.
 - ▶ **Examples:** the set of all machines which compute a certain function, the set of all terminating programs, the set of all programs which never output a certain “bad” string.

Theorem (Rice's Theorem) $s \wedge d \rightarrow t$

Every semantic decidable language $\mathcal{L}$ is trivial, i.e. either $\mathcal{L} = \emptyset$ or $\mathcal{L} = \{0,1\}^$.*

$$\neg t \rightarrow \neg(s \wedge d)$$

=

$$\neg t \rightarrow \neg s \vee \neg d$$

How to Prove a Problem to be *Computable*?

- ▶ One can of course **construct a TM** which computes the **given function** or that decides the given language.
- ▶ One can describe, in a more informal way, **an algorithm** which itself computes the function or decides the language.
 - ▶ It is of course crucial to be sure that all steps the algorithm perform, i.e., **all instructions**, are elementary, or at least compute functions which **are already known to be computable**.
 - ▶ In doing so, one can use other algorithms as “subroutines”.
 - ▶ Evaluating the performances of algorithms defined this way is possible, but often not so precisely.

How to Prove a Problem to be *Uncomputable*?

- ▶ You can **prove that** the problem under consideration, call it $\mathcal{L}$ is **at least as hard** as a problem you already know to be undecidable, call it $\mathcal{G}$

- ▶ You have to show that there is a computable way ϕ of turning strings in $\{0, 1\}^*$ into strings in $\{0, 1\}^*$ in such a way that

$$s \in \mathcal{G} \Leftrightarrow \phi(s) \in \mathcal{L}$$

- ▶ This way, any hypothetical algorithm for $\mathcal{L}$ would be turned into one for $\mathcal{G}$, which cannot exist however.
- ▶ You can use results like **Rice's Theorem**.